

Table of Contents

- Titlei
- ACKNOWLEDGEMENT ii
- ABSTRACT / EXECUTIVE SUMMARYii
- ABBREVIATIONS ii
- 1. INTRODUCTION 1
 - 1.1 Background 1
 - 1.2 Problem Statement 1
 - 1.3 Objectives 1
 - 1.4 Scope of the Project 1
- 2. REQUIREMENT ANALYSIS 2
 - 2.1 Functional Requirements 2
 - 2.2 Non-Functional Requirements 2
- 3. EXPECTED OUTPUT 3
- 4. SYSTEM DESIGN 4-5
 - 4.1 Architecture 4
 - 4.2 Database Design 4
 - 4.2.1 ER Diagram (Conceptual) 4
 - 4.2.2 ER To Table 5
 - 4.2.3 Relational Schema (Table Definitions) 5
 - 4.2.4 Relationship Analysis (Participation) 5
- 5. IMPLEMENTATION DETAILS 6
 - 5.1 Technologies Used 5
 - 5.2 Key Modules & Functionality 5
- 6. TESTING (BRIEF OVERVIEW) 7
- 7. RESULTS & SCREENSHOTS 8-9
- 8. CONCLUSION 10
- 9. FUTURE WORK & ENHANCEMENTS 10
- 10 . REFERENCES 10

Title:

CarRentPro: A Complete Online Car Rental Management System

CarRentPro

This document presents a comprehensive overview of CarRentPro, an end-to-end online vehicle rental management system designed to streamline rental processes for customers and administrators. It covers system objectives, database design including ER diagrams and normalization, key functionality implementations, and system validation through execution snapshots. The final sections provide conclusions on the system's capabilities and reference resources to support the project development

ACKNOWLEDGEMENT

We would like to thank our mentors and faculty members for their constant support throughout the development of this project "CarRentPro". Their guidance helped us overcome challenges and achieve the desired objectives.

ABSTRACT / EXECUTIVE SUMMARY

"CarRentPro" is a Vehicle Rental Management System designed to facilitate customers in reserving vehicles online while offering a separate admin panel for management tasks. Customers can browse available vehicles, reserve them, view rentals, and manage payments, while admins handle reservation approvals, vehicle returns, and oversee financial transactions.

ABBREVIATIONS

- ER: Entity-Relationship
- DBMS: Database Management System
- UI: User Interface
- SQL: Structured Query Language
- CRUD: Create, Read, Update, Delete

1. INTRODUCTION

1.1 Background

The project aims to automate and digitize the car rental process, replacing manual reservations and paper-based record keeping with a seamless online system.

1.2 Problem Statement

Existing car rental systems often lack seamless integration of reservation, pickup, return, and payment workflows, causing inefficiencies and user dissatisfaction.

1.3 Objectives

- Build a user-friendly platform for customers.
- Allow admins to efficiently manage reservations, pickups, returns, and payments.
- Enable easy vehicle availability management.
- Generate clear rental and payment records.

1.4 Scope of the Project

1.4.1 Functional Scope

- Customer Registration/Login
- Vehicle Browsing and Reservation
- Admin Approval of Reservations
- Vehicle Pickup and Return Management
- Online Payment Handling
- User Profile Management

1.4.2 Technical Scope

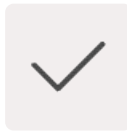
- Python (Flask Framework)
- MySQL Database
- Frontend using HTML, CSS
- Session Management for Authentication

1.4.3 Limitations (Out of Scope)

- Real-time payment gateway integration (mock QR used)
- Real GPS vehicle tracking
- Automatic fine calculation for delays

2. REQUIREMENT ANALYSIS

2.1 Functional Requirements

-  Login and Registration System
-  Browse and Search Vehicles
-  Reserve Vehicles
-  Admin Approval/Cancel Reservations
-  Confirm Car Pickups and Returns
-  Manage Payments (Customer and Admin View)

2.2 Non-Functional Requirements

- 1User-friendly Interface
- 2Secure Session Management
- 3Quick Load Time
- 4Responsive Design
- 5Proper Validation on Forms

3. EXPECTED OUTPUT

	Successful user registration and login Users can create accounts and securely access the system
	Listing of available vehicles Customers can browse and search the vehicle inventory
	Successful reservation and admin approval Reservation process with administrative oversight
	Tracking of vehicle status Monitoring vehicles as available, reserved, or rented
	Payment handling and transaction logging Complete financial transaction management

4. SYSTEM DESIGN

4.1 Architecture

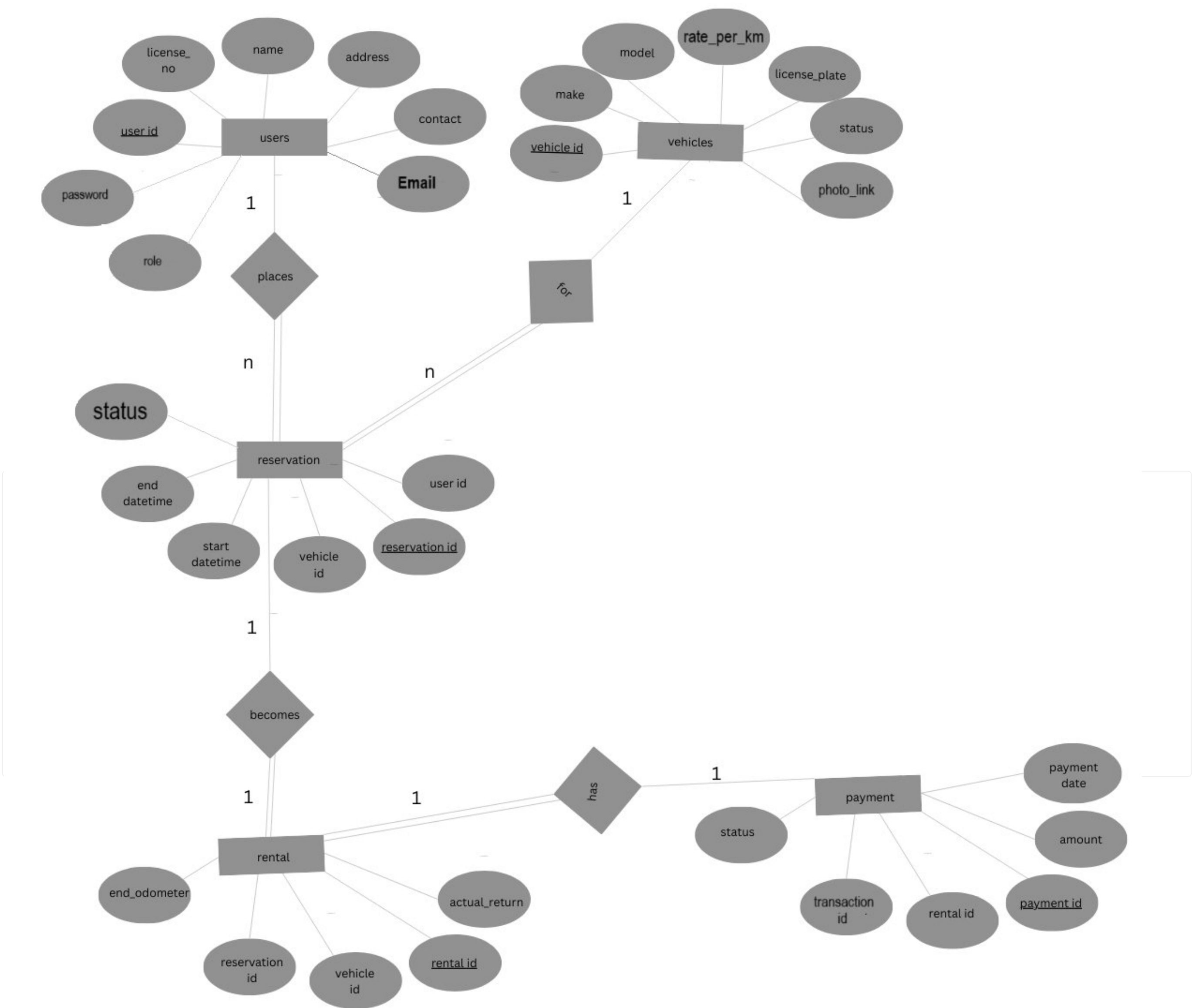
The CarRentPro system implements:

- Client-Server Model
- MVC (Model-View-Controller) Pattern using Flask



4.2 Database Design

4.2.1 ER Diagram (Conceptual)



4.2.2 ER To Table

Entities converted to Tables:

- Users
- Vehicles
- Reservation
- Rentals
- Payments

4.2.3 Relational Schema (Table Definitions)

Users(id, name, email, address, contact, license_no, password, role)

Vehicle(vehicle_id, make, model, odometer_reading, rate_per_km, status, photo_link)

Reservation(reservation_id, customer_id, vehicle_id, start_datetime, end_datetime, status)

Rental(rental_id, reservation_id, start_odometer, end_odometer, actual_pickup, actual_return, returned)

Payments(payment_id, rental_id, amount, transaction_id, payment_date, status)

4.2.4 Relationship Analysis (Participation)

- Users to Reservation: 1 to Many
- Vehicles to Reservation: 1 to Many
- Reservation to Rentals: 1 to 1
- Rentals to Payments: 1 to 1

5. IMPLEMENTATION DETAILS

5.1 Technologies Used



Backend:
Python (Flask)

Server-side programming and application logic



Frontend:
HTML, CSS, Bootstrap, JavaScript

User interface and client-side functionality



Database:
MySQL

Data storage and management



Server:
Localhost (Flask development server)

Application hosting environment

5.2 Key Modules & Functionality

Customer Interface

- Register, Login, Browse, Reserve, Pay

Admin Interface

- View, Approve, Manage Returns, Confirm Payments

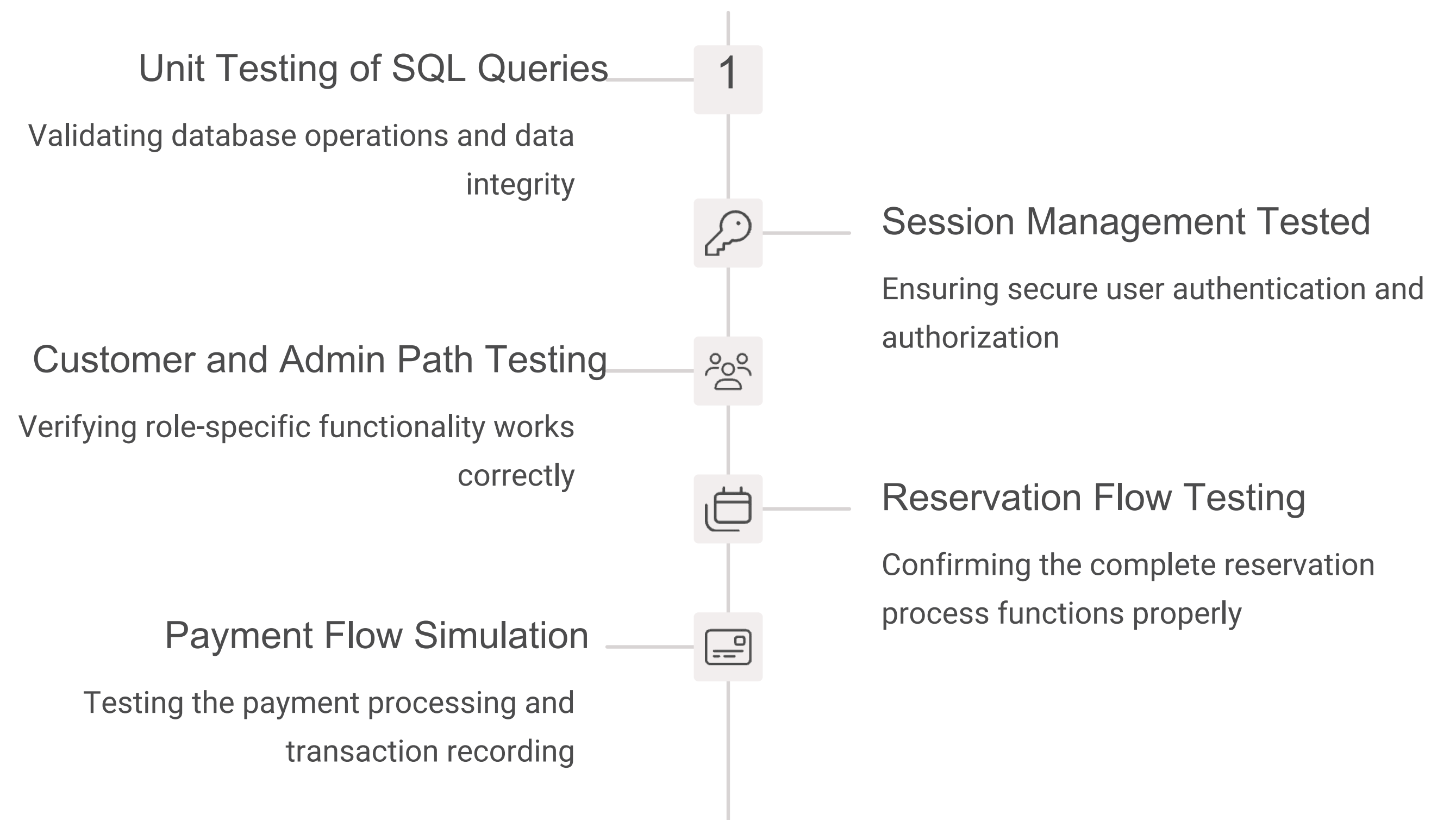
Vehicle Availability Management

- Track and update vehicle status

Rental and Payment Record Management

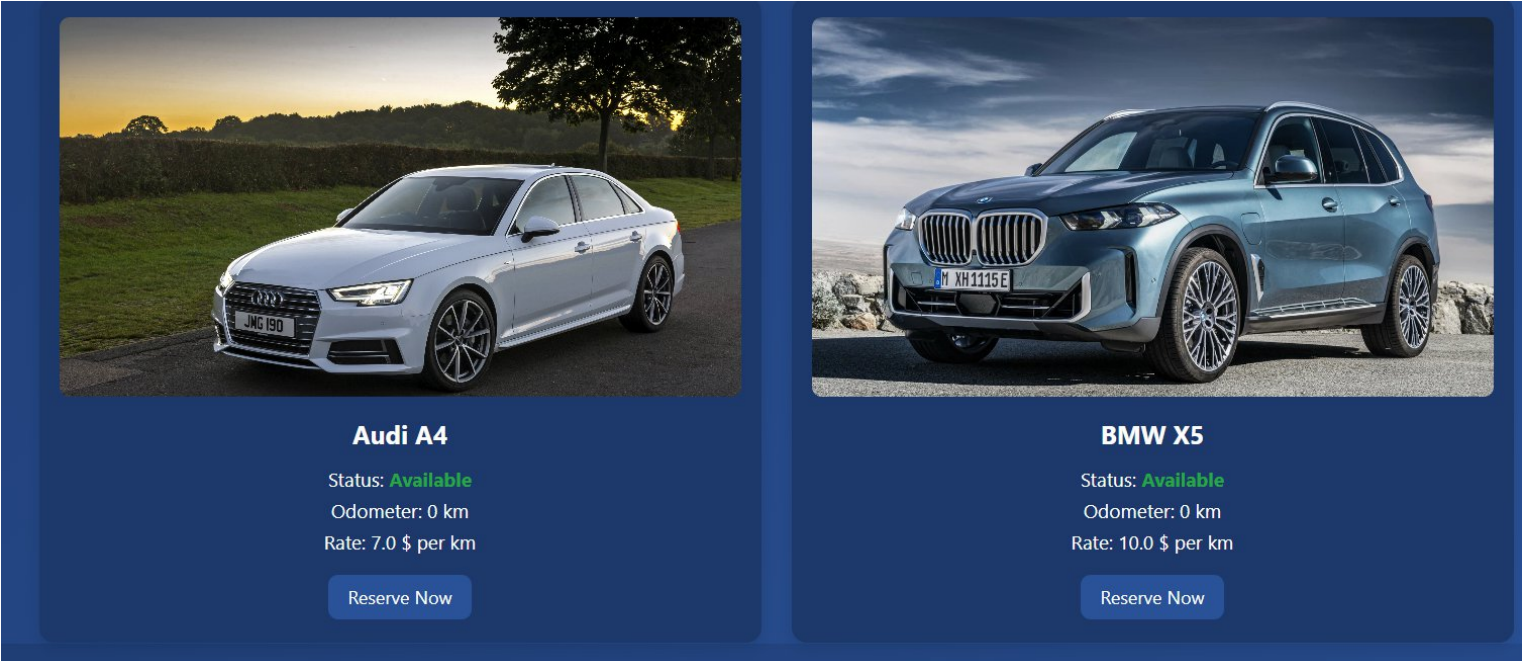
- Document transactions and generate reports

6. TESTING (BRIEF OVERVIEW)



7. RESULTS & SCREENSHOTS(Queries)

```
sql = """
SELECT
    vehicle_id,
    make,
    model,
    rate_per_km,
    odometer_reading,
    photo_link,
    status
FROM vehicle
ORDER BY make, model
"""
```



```
sql = """
SELECT r.reservation_id,
       u.name AS user_name,
       u.email,
       u.contact,
       r.start_datetime,
       r.end_datetime,
       v.make,
       v.model
FROM reservation r
JOIN users u ON r.customer_id = u.id
JOIN vehicle v ON r.vehicle_id = v.vehicle_id
WHERE r.status = 'pending'
ORDER BY r.start_datetime ASC
"""
```

Reserve Requests

Customer Name	Email	Phone	Start Date	End Date	Vehicle	Action
Suri	suraj@gmail.com	98989898989	2025-04-28 04:30:00	2025-04-30 05:30:00	Toyota Camry	<button>Approve</button>

```
sql = """
SELECT r.reservation_id,
       u.name AS user_name,
       u.email,
       u.contact,
       r.start_datetime,
       r.end_datetime,
       v.make,
       v.model
FROM reservation r
JOIN users u ON r.customer_id = u.id
JOIN vehicle v ON r.vehicle_id = v.vehicle_id
WHERE r.status = 'reserved'
AND (
    LOWER(u.name) LIKE %s
    OR LOWER(u.email) LIKE %s
    OR u.contact LIKE %s
)
ORDER BY r.start_datetime DESC
"""

cursor.execute(sql, (like_term, like_term, like_term))
else:
    sql = """
    SELECT r.reservation_id,
           u.name AS user_name,
           u.email,
           u.contact,
           r.start_datetime,
           r.end_datetime,
           v.make,
           v.model
    FROM reservation r
    JOIN users u ON r.customer_id = u.id
    JOIN vehicle v ON r.vehicle_id = v.vehicle_id
    WHERE r.status = 'reserved'
    ORDER BY r.start_datetime DESC
    """
    cursor.execute(sql)
```

Car Pick Up

Search by name, email or phone  Search

Customer Name	Email	Phone	Start Date	End Date	Vehicle	Action
Suri	suraj@gmail.com	98989898989	2025-04-28 04:30:00	2025-04-30 05:30:00	Toyota Camry	<button>Confirm Rent</button> <button>Cancel</button>

```
sql = """
SELECT r.reservation_id,
       rental.rental_id,
       u.name AS user_name,
       u.email,
       v.make,
       v.model,
       r.start_datetime,
       r.end_datetime,
       v.odometer_reading AS start_odometer
FROM reservation r
JOIN rental rental ON rental.reservation_id = r.reservation_id
JOIN users u ON r.customer_id = u.id
JOIN vehicle v ON r.vehicle_id = v.vehicle_id
WHERE r.status = 'rented'
      AND rental.returned = FALSE
      AND (
            LOWER(u.name) LIKE %s
          OR LOWER(u.email) LIKE %s
          OR u.contact LIKE %s
        )
ORDER BY r.start_datetime DESC
"""

cursor.execute(sql, (like, like, like))

else:
    sql = """
    SELECT r.reservation_id,
           rental.rental_id,
           u.name AS user_name,
           u.email,
           v.make,
           v.model,
           r.start_datetime,
           r.end_datetime,
           v.odometer_reading AS start_odometer
    FROM reservation r
    JOIN rental rental ON rental.reservation_id = r.reservation_id
    JOIN users u ON r.customer_id = u.id
    JOIN vehicle v ON r.vehicle_id = v.vehicle_id
    WHERE r.status = 'rented'
          AND rental.returned = FALSE
    ORDER BY r.start_datetime DESC
    """
    cursor.execute(sql)
```

Return Section



Customer Name	Email	Vehicle	Start	End	Start Odo	Action
Suri	suraj@gmail.com	Toyota Camry	2025-04-28 04:30:00	2025-04-30 05:30:00	1000	Confirm Return

```
cursor.execute("""
SELECT rent.rental_id,
       v.make, v.model,
       rent.actual_pickup,
       rent.actual_return,
       rent.start_odometer,
       rent.end_odometer,
       v.rate_per_km AS rate_per_km,
       p.payment_id
FROM rental rent
JOIN reservation r ON rent.reservation_id = r.reservation_id
JOIN payments p ON rent.rental_id = p.rental_id
JOIN vehicle v ON r.vehicle_id = v.vehicle_id
WHERE r.customer_id = %s
      AND rent.returned= TRUE
      AND p.status = 'unpaid'
""", (session['user_id'],))
rows = cursor.fetchall()

cursor.close()
db.close()
```

Vehicle	Pickup	Return	Distance (km)	Amount (\$)	
Toyota Camry	2025-04-28 03:28:27	2025-04-29 03:42:00	500	2500.00	Paid?

```
sql = """
SELECT
  u.name AS user_name,
  u.email AS email,
  CONCAT(v.make, ' ', v.model) AS vehicle,
  rent.actual_pickup,
  rent.actual_return,
  (rent.end_odometer - rent.start_odometer) AS total_travelled,
  p.amount,
  p.transaction_id,
  p.status
FROM payments p
JOIN rental rental ON p.rental_id = rent.rental_id
JOIN reservation r ON rent.reservation_id = r.reservation_id
JOIN users u ON r.customer_id = u.id
JOIN vehicle v ON r.vehicle_id = v.vehicle_id
WHERE
  LOWER(u.name) LIKE %s
OR LOWER(u.email) LIKE %s
OR p.transaction_id LIKE %s
ORDER BY rent.actual_return DESC
"""
```

All Payments



Customer	Email	Vehicle	Pickup	Return	Distance	Amount	Txn ID	Status
Suri	suraj@gmail.com	Toyota Camry	2025-04-28 03:28:27	2025-04-29 03:42:00	500	2500.00	22300003233	Paid

8. CONCLUSION

The "CarRentPro" project successfully creates a complete cycle of renting, reserving, picking up, returning, and paying for vehicles. Both customers and admins experience smooth workflows, enhancing transparency and efficiency.

9. FUTURE WORK & ENHANCEMENTS

	Online Payment Gateway Integration Replace mock payment with real-time processing
	OTP Authentication during login Enhance security with two-factor verification
	Add Admin Analytics Dashboard Provide business intelligence and reporting
	Push Notifications for Reservation Updates Improve customer communication
	Dynamic Pricing Based on Peak Hours Optimize revenue with demand-based pricing

10. REFERENCES

- Flask Documentation: <https://flask.palletsprojects.com/>
- MySQL Documentation: <https://dev.mysql.com/doc/>
- Bootstrap Official Documentation: <https://getbootstrap.com/>
- W3Schools (HTML/CSS/JS Basics): <https://www.w3schools.com/>
- FontAwesome Icons: <https://fontawesome.com/>
- Python (Language Basics): <https://www.python.org/>